

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR LEGAL CASE & CLIENT MANAGEMENT SYSTEM

A Database-Driven Web Application for Managing Clients, Lawyers, Judges, Cases,
Hearings and Payments

Project Information	Details
Student Name	<u>M.Gagan Sri Varma</u>
Roll Number	<u>25WU0102150</u>
Course	Database Management Systems (DBMS)
Faculty Name	<u>Kiran Mayee Mam</u>
Academic Year	2025–2029
Project Type	DBMS Course Project

Woxsen University

CSE – AIML

2. Abstract

The **Legal Case & Client Management System** is a database-driven application designed to organize and manage legal information in a structured relational system. The system maintains client records, lawyer information, judge details, legal cases, hearing schedules, payment records, and lawyer–case assignments. MySQL is used as the relational database, while a Node.js and Express.js backend provides REST-based access to the database and the frontend provides an interactive interface for users.

The system supports core CRUD operations, relational queries, case tracking, hearing management, payment tracking, search and filtering, and dashboard-level reporting. Primary keys, foreign keys, relational constraints, and normalized table structures are used to preserve data integrity and reduce unnecessary duplication. The application is intended to provide a clear and efficient workflow for maintaining legal case information while demonstrating practical DBMS concepts in a full-stack environment.

3. Introduction and Problem Statement

3.1 Introduction

Legal organizations manage a large amount of interconnected information. A single client may have one or more legal cases, cases may involve different lawyers, hearings are associated with judges and courts, and payments are linked to clients and cases. Managing these records independently can result in duplicate data, missing information, difficulty in tracking case progress, and inconsistent records.

This project models these relationships using a relational database and a connected web interface. The database separates major entities into dedicated tables while foreign-key relationships connect related records. The frontend and backend provide a practical interface through which authorized users can view and manage the database.

3.2 Problem Statement

A legal case management environment requires a reliable system to maintain client details, lawyer and judge information, case records, hearing schedules, and payment information. Manual or disconnected record management makes searching, updating, and maintaining relationships between records difficult. The proposed system addresses this problem by providing a centralized relational database and a user-friendly web interface that supports CRUD operations and relationship-based information retrieval.

4. Objectives and Scope

4.1 Objectives

- Maintain client, lawyer, judge, case, hearing, payment, and lawyer–case assignment records in MySQL.
- Establish relationships between entities using primary and foreign keys.
- Provide database-backed CRUD operations through the user interface.
- Track legal cases, their status, filing dates, courts, and descriptions.
- Manage hearing dates, times, judges, locations, and notes.
- Record client and case-related payments.
- Support assignment of multiple lawyers to cases through the Works_On relationship.
- Provide search, filtering, dashboard statistics, and reporting-oriented queries.
- Maintain consistency and referential integrity using relational constraints.

4.2 Scope

The scope covers a centralized legal case database and a web-based management interface. The current system focuses on operational database management and does not claim to automate legal decision-making, court proceedings, or legal advice. Authentication, role-based access control, cloud deployment, automated notifications, and advanced analytics may be added as future enhancements.

5. Software and Hardware Requirements

Category	Requirement
Operating System	macOS / Windows / Linux
Frontend	HTML, CSS, JavaScript with Bootstrap 5 or equivalent responsive UI
Backend	Node.js and Express.js
Database	MySQL
DB Connectivity	mysql2
Configuration	dotenv
Development Environment	VS Code and MySQL Workbench
Browser	Modern Chrome / Firefox / Safari / Edge
Recommended Hardware	Dual-core processor, 8 GB RAM, 10 GB free storage

6. ER Diagram

The Entity–Relationship (ER) diagram represents the major entities in the Legal Case & Client Management System and the relationships between them. The model includes Client, Lawyer, Judge, LegalCase, Hearing, Payment, and Works_On. Client and LegalCase are linked through client_id; hearings are connected to cases and judges; payments connect clients and cases; and Works_On represents the many-to-many relationship between lawyers and cases.

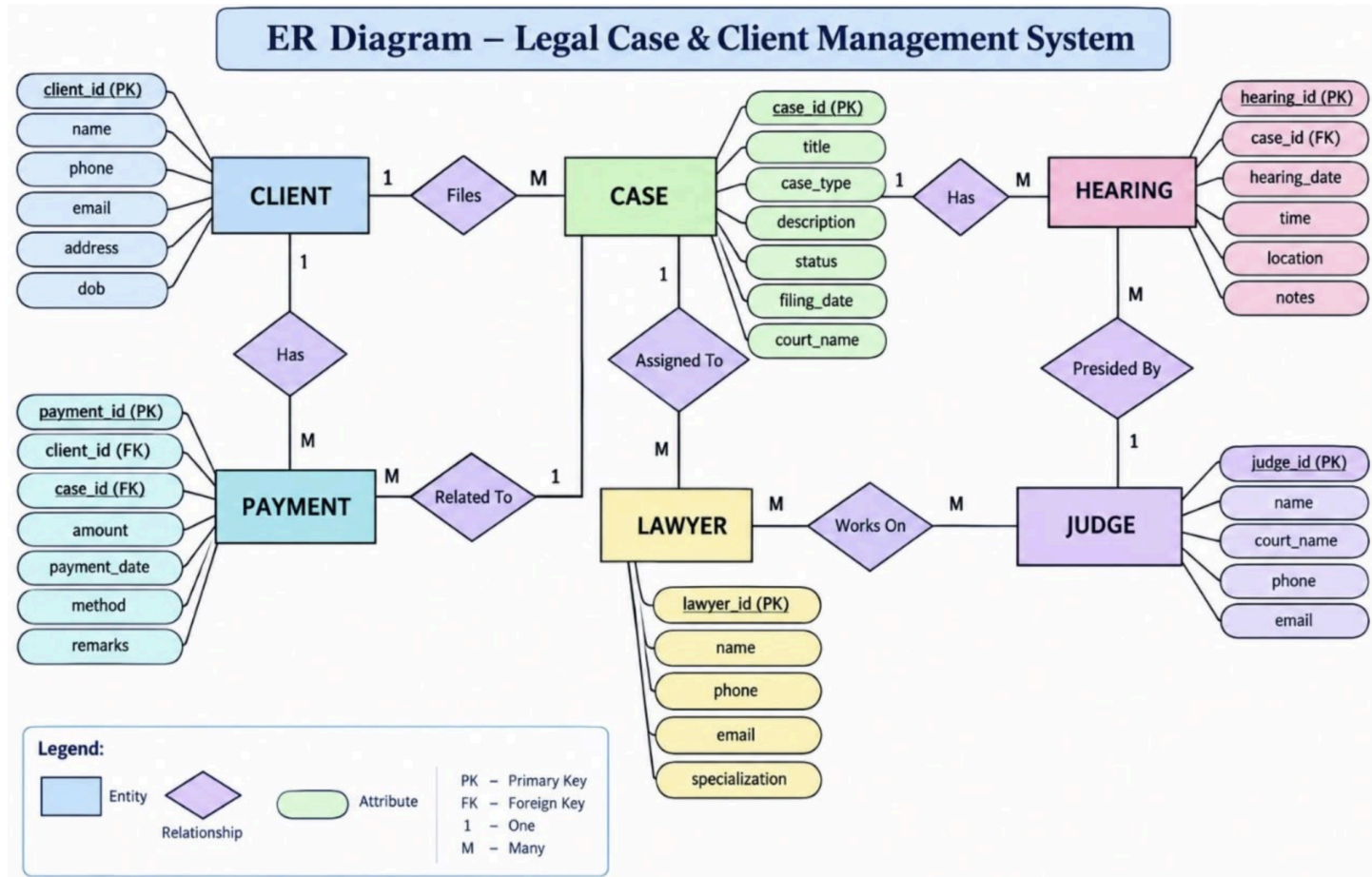


Figure 1. ER diagram placeholder — insert the finalized ER diagram here.

7. Relational Schema and Normalization

7.1 Relational Schema

Relation	Primary Key	Foreign Keys
CLIENT	client_id	—
LAWYER	lawyer_id	—
JUDGE	judge_id	—
LEGALCASE	case_id	client_id → CLIENT
HEARING	hearing_id	case_id → LEGALCASE; judge_id → JUDGE
PAYMENT	payment_id	client_id → CLIENT; case_id → LEGALCASE
WORKS_ON	lawyer_id, case_id	lawyer_id → LAWYER; case_id → LEGALCASE

7.2 Normalization

The database is organized to reduce redundancy and maintain consistent relationships. Each entity is represented by a dedicated relation, while related information is connected using foreign keys. Attributes are atomic and there are no repeating groups, supporting First Normal Form (1NF). Single-column primary keys are used for the major entities, while Works_On uses a composite key to represent a many-to-many relationship. Non-key attributes are stored with the entity they describe, supporting Third Normal Form (3NF).

Normal Form	Applied Principle
1NF	Atomic attributes and no repeating groups.
2NF	Non-key attributes depend on the complete primary key; Works_On uses a composite key only for the
3NF	Non-key attributes describe their respective entities and are separated into appropriate relations.

relationship.

8. Data Dictionary

8.1 CLIENT

Column	Type	Constraint / Meaning
client_id	INT	PK; client identifier; AUTO_INCREMENT
name	VARCHAR(100)	Client name; NOT NULL
phone	VARCHAR(15)	Contact number
email	VARCHAR(100)	Email address
address	VARCHAR(255)	Client address
dob	DATE	Date of birth

8.2 LAWYER

Column	Type	Constraint / Meaning
lawyer_id	INT	PK; lawyer identifier; AUTO_INCREMENT
name	VARCHAR(100)	Lawyer name; NOT NULL
phone	VARCHAR(15)	Contact number
email	VARCHAR(100)	Email address
specialization	VARCHAR(100)	Area of legal specialization

8.3 JUDGE

Column	Type	Constraint / Meaning
judge_id	INT	PK; judge identifier; AUTO_INCREMENT
name	VARCHAR(100)	Judge name; NOT NULL
court_name	VARCHAR(100)	Associated court
phone	VARCHAR(15)	Contact number
email	VARCHAR(100)	Email address

8.4 LEGALCASE

Column	Type	Constraint / Meaning
case_id	INT	PK; case identifier; AUTO_INCREMENT

Column	Type	Constraint / Meaning
client_id	INT	FK → Client
title	VARCHAR(150)	Case title
case_type	VARCHAR(100)	Type/category of case
description	TEXT	Case description
status	VARCHAR / ENUM	Case status such as PENDING, ONGOING, COMPLETED
filing_date	DATE	Date the case was filed
court_name	VARCHAR(100)	Court associated with the case

8.5 HEARING

Column	Type	Constraint / Meaning
hearing_id	INT	PK; hearing identifier; AUTO_INCREMENT
case_id	INT	FK → LegalCase
judge_id	INT	FK → Judge
hearing_date	DATE	Scheduled hearing date
hearing_time	TIME	Scheduled hearing time
location	VARCHAR(150)	Hearing location
notes	TEXT	Additional hearing notes

8.6 PAYMENT

Column	Type	Constraint / Meaning
payment_id	INT	PK; payment identifier; AUTO_INCREMENT
client_id	INT	FK → Client
case_id	INT	FK → LegalCase
amount	DECIMAL(10,2)	Payment amount
payment_date	DATE	Payment date
method	VARCHAR(50)	Payment method
remarks	VARCHAR(255)	Payment remarks

8.7 WORKS_ON

Column	Type	Constraint / Meaning
lawyer_id	INT	PK/FK → Lawyer
case_id	INT	PK/FK → LegalCase
	Composite primary key	(lawyer_id, case_id); represents lawyer–case assignment

9. SQL Commands Used (DDL and DML)

9.1 DDL

The database structure was created using Data Definition Language (DDL) commands. The following representative statements illustrate database creation and table definition. The complete implementation should use the project SQL script as the authoritative source.

```
CREATE DATABASE LegalCaseDB;
USE LegalCaseDB;

CREATE TABLE Client (
  client_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  phone VARCHAR(15),
  email VARCHAR(100),
  address VARCHAR(255),
  dob DATE
);
```

9.2 DML

DML commands are used to insert, update, delete, and retrieve records. Representative operations include adding client records, updating case information, assigning lawyers to cases, recording hearings, and recording payments.

```
INSERT INTO Client (name, phone, email, address, dob)
VALUES ('Sample Client', '9876543210', 'sample@example.com', 'Hyderabad', '2000-01-15');

UPDATE LegalCase
SET status = 'ONGOING'
WHERE case_id = 1;

DELETE FROM Payment
WHERE payment_id = 20;
```

9.3 Sample Output

Operation	Result
INSERT	A client record is inserted into the Client relation.
UPDATE	The selected legal case status is changed.
DELETE	The selected payment record is removed subject to relational constraints.

10. Queries with Outputs

The following queries demonstrate important database operations and relationships in the project.

10.1 View all tables

```
SHOW TABLES;
```

Expected Output: Client, Lawyer, Judge, LegalCase, Hearing, Payment, Works_On.

10.2 Client and Case relationship

```
SELECT c.name, lc.case_id, lc.title,
lc.status FROM Client c
JOIN LegalCase lc ON c.client_id = lc.client_id
ORDER BY c.client_id;
```

Output: Client names are displayed with their linked legal cases and current status.

10.3 Case, Hearing and Judge relationship

```
SELECT lc.title, h.hearing_date,
h.hearing_time, j.name AS judge_name,
h.location
```

```
FROM LegalCase lc
JOIN Hearing h ON lc.case_id = h.case_id
JOIN Judge j ON h.judge_id = j.judge_id
ORDER BY h.hearing_date;
```

Output: Cases are displayed together with scheduled hearing details and the assigned judge.

10.4 Client, Case and Payment relationship

```
SELECT c.name, lc.title, p.amount, p.payment_date,
p.method FROM Client c
JOIN Payment p ON c.client_id =
p.client_id JOIN LegalCase lc ON p.case_id
= lc.case_id ORDER BY p.payment_date;
```

Output: Client payment records are displayed with the associated legal case.

10.5 Lawyer and Case assignment

```
SELECT l.name AS lawyer_name, lc.title AS case_title
FROM Lawyer l
JOIN Works_On w ON l.lawyer_id = w.lawyer_id
JOIN LegalCase lc ON w.case_id = lc.case_id
ORDER BY l.lawyer_id;
```

Output: Lawyers are displayed with the legal cases to which they are assigned.

10.6 Dashboard-style case statistics

```
SELECT status, COUNT(*) AS total_cases
FROM LegalCase
GROUP BY status;
```

Output: The query returns the number of cases in each recorded status.

11. UI Design and Screenshots

The interface is designed as a focused legal operations dashboard. It provides dashboard statistics, client and lawyer management, judge records, case management, hearing scheduling, payment tracking, search/filtering, CRUD actions, confirmation dialogs, loading states, and controlled error messages. The visual design should remain minimal and professional, with a subtle premium green accent and restrained animation.

11.1 Dashboard Screenshot

Dashboard showing total clients, lawyers, cases, upcoming hearings and payment statistics.

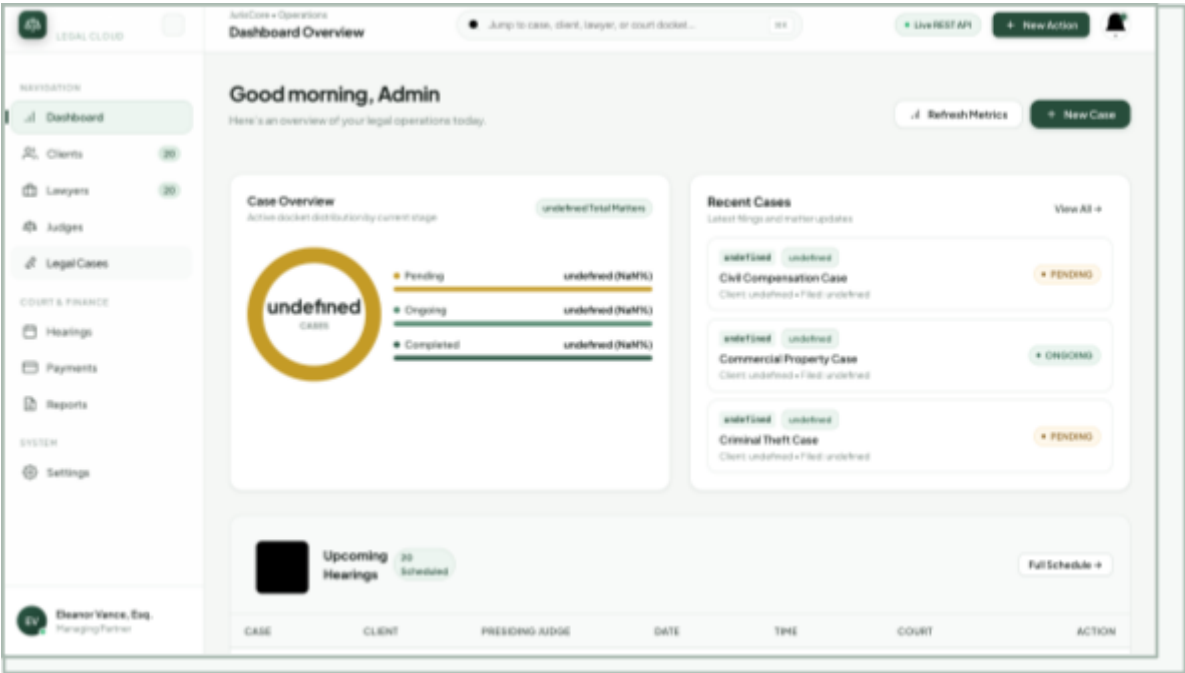
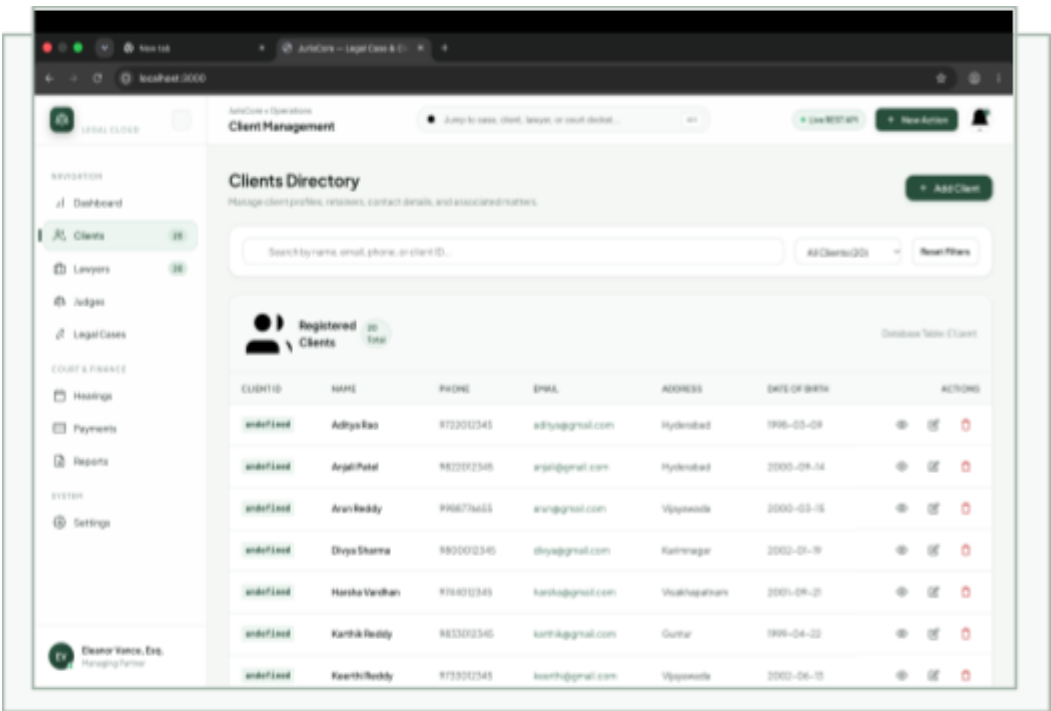


Figure — 11.1 Dashboard Screenshot.

11.2 Client Management Screenshot



Client listing with search, add, edit and delete actions.

Figure — 11.2 Client Management Screenshot.

11.3 Lawyer Management Screenshot

Lawyer records with specialization and assigned-case information.

JurisCore

LEGAL CLOUD

NAVIGATION

Dashboard

Clients20

Lawyers20

Judges

Legal Cases

COURT & FINANCE

Hearings

Payments

Reports

SYSTEM

Settings

EV

Eleanor Vance, Esq.

Managing Partner

JurisCore • Operations

Lawyer Registry

Jump to case, client, lawyer, or court docket...

Live REST API

New Action

Legal Counsel Registry

20 Counsel

Database Table: Lawyer

Search by lawyer name, bar number, or email...

All Specializations (20)

Reset Filters

LAWYER ID	NAME	SPECIALIZATION	PHONE	EMAIL	ACTIVE MATTERS	ACTIONS
LAW-2001	Eleanor Vance, Esq. Bar: NY-492019	Corporate Law	+1 (555) 910-4401	eleanor.vance@juriscore.law	2 cases	
LAW-2002	Marcus Thorne, Esq. Bar: MA-331092	Criminal Law	+1 (555) 910-4402	marcus.thorne@juriscore.law	1 cases	
LAW-2003	Siddharth Mehta, Esq. Bar: CA-882190	Cyber Law	+1 (555) 910-4403	siddharth.mehta@juriscore.law	1 cases	
LAW-2004	Claire DeWitt, Esq. Bar: IL-672901	Civil Law	+1 (555) 910-4404	claire.dewitt@juriscore.law	1 cases	
LAW-2005	Jonathan Sterling, Esq. Bar: NY-501934	Property Law	+1 (555) 910-4405	jonathan.sterling@juriscore.law	1 cases	
LAW-2006	Amara Okonjo, Esq.	Tax Law	+1 (555) 910-4406	amara.okonjo@juriscore.law	1 cases	

JurisCore

LEGAL CLOUD

NAVIGATION

Dashboard

Clients20

Lawyers20

Judges

Legal Cases

COURT & FINANCE

Hearings

Payments

Reports

SYSTEM

Settings

EV

Eleanor Vance, Esq.

Managing Partner

JurisCore • Operations

Lawyer Registry

Jump to case, client, lawyer, or court docket...

Live REST API

New Action

LAW-2006	Amara Okonjo, Esq. Bar: DC-109283	Tax Law	+1 (555) 910-4406	amara.okonjo@juriscore.law	1 cases	
LAW-2007	Vivian H. Hayes, Esq. Bar: CT-782012	Family Law	+1 (555) 910-4407	vivian.hayes@juriscore.law	1 cases	
LAW-2008	Julian Rivera, Esq. Bar: TX-440918	Corporate Law	+1 (555) 910-4408	julian.rivera@juriscore.law	1 cases	
LAW-2009	Genevieve Moreau, Esq. Bar: WA-661209	Cyber Law	+1 (555) 910-4409	genevieve.moreau@juriscore.law	1 cases	
LAW-2010	Devon Fitzpatrick, Esq. Bar: PA-309184	Criminal Law	+1 (555) 910-4410	devon.fitz@juriscore.law	3 cases	
LAW-2011	Aisha Al-Hashimi, Esq. Bar: FL-902384	Property Law	+1 (555) 910-4411	aisha.hashimi@juriscore.law	4 cases	
LAW-2012	Lucas Brennan, Esq. Bar: CO-551982	Tax Law	+1 (555) 910-4412	lucas.brennan@juriscore.law	1 cases	
LAW-2013	Miriam Chen, Esq. Bar: CA-994018	Civil Law	+1 (555) 910-4413	miriam.chen@juriscore.law	1 cases	
LAW-2014	Harrison Brooks, Esq. Bar: NY-281903	Corporate Law	+1 (555) 910-4414	harrison.brooks@juriscore.law	1 cases	
LAW-2015	Nadine Dubois, Esq. Bar: MA-774012	Family Law	+1 (555) 910-4415	nadine.dubois@juriscore.law	1 cases	
LAW-2016	Rajesh Kothari, Esq. Bar: IL-449102	Criminal Law	+1 (555) 910-4416	rajesh.kothari@juriscore.law	3 cases	
LAW-2017	Seraphina Locke, Esq. Bar: UT-188904	Property Law	+1 (555) 910-4417	seraphina.locke@juriscore.law	3 cases	

Figure — 11.3 Lawyer Management Screenshot.

11.4 Judge Management Screenshot

Judge records with court and contact information.

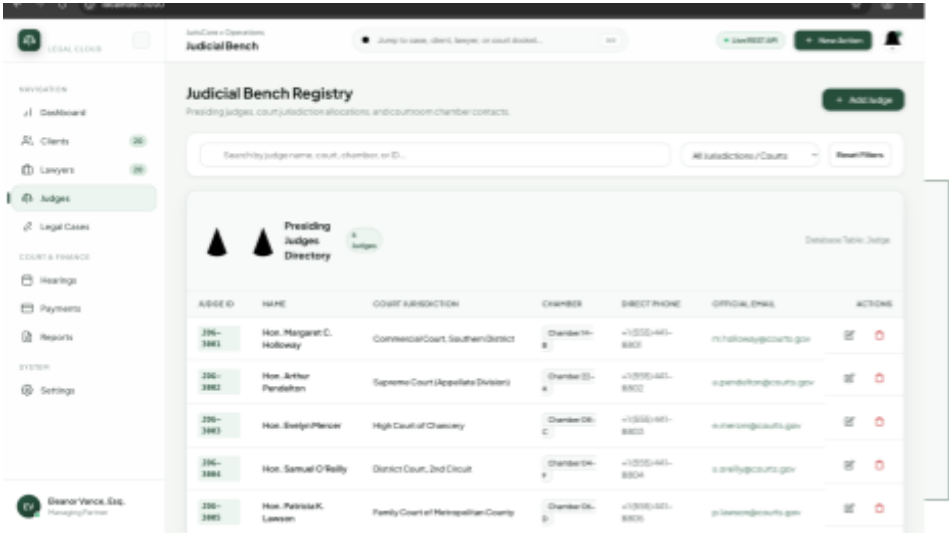


Figure — 11.4 Judge Management Screenshot.

11.5 Case Management Screenshot

Case list and detailed case profile including client, lawyer, hearing and payment information.

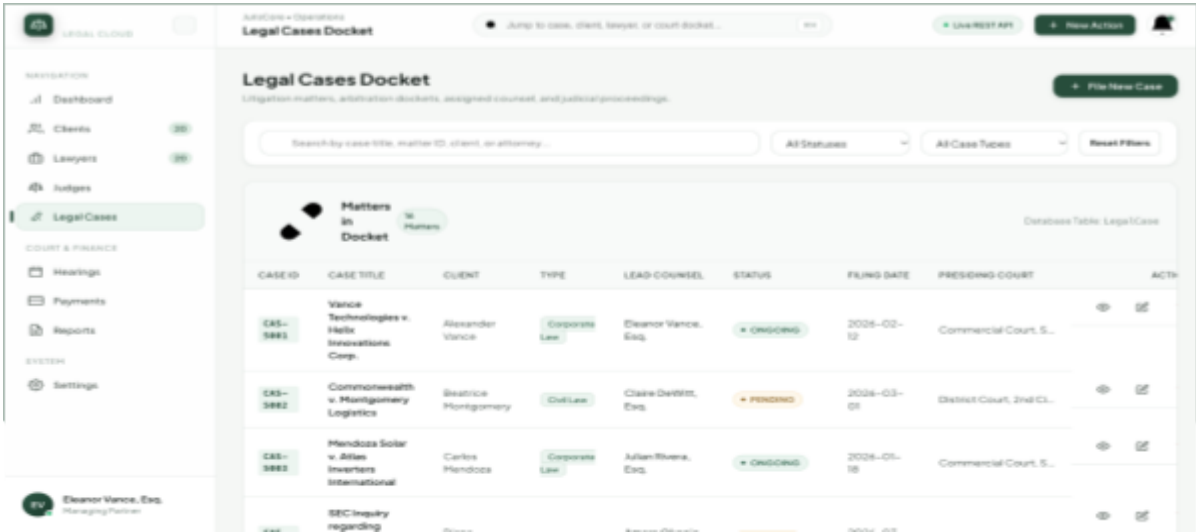


Figure — 11.5 Case Management Screenshot.

11.6 Hearing Management Screenshot

Hearing schedule showing case, judge, date, time and location.

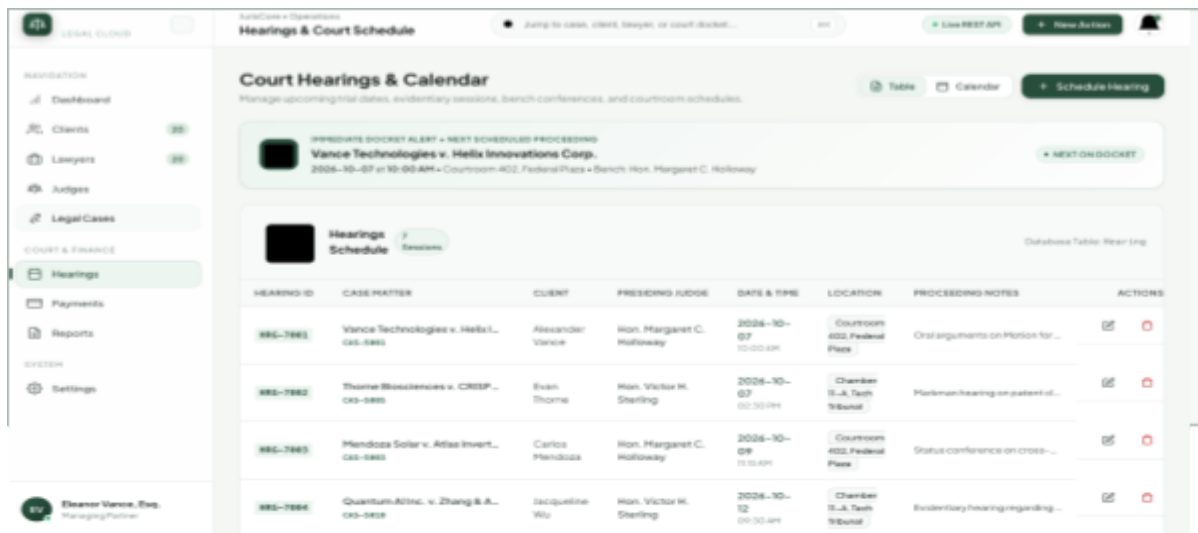


Figure — 11.6 Hearing Management Screenshot.

11.7 Payment Management Screenshot

Payment records with amount, date, method and remarks.

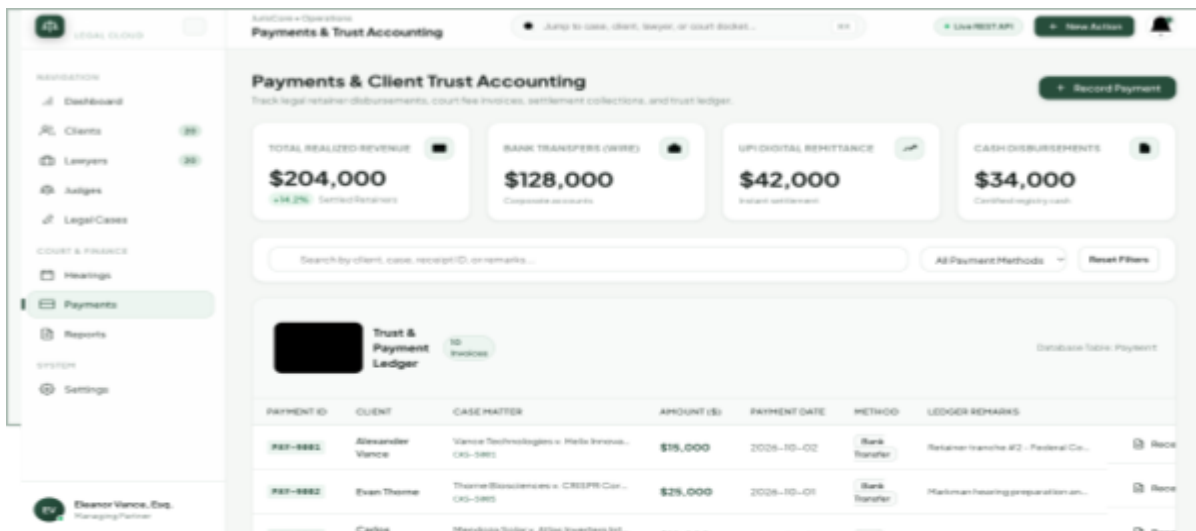


Figure — 11.7 Payment Management Screenshot.

11.8 Reports Screenshot

Reports and analytical views generated from relational database queries.

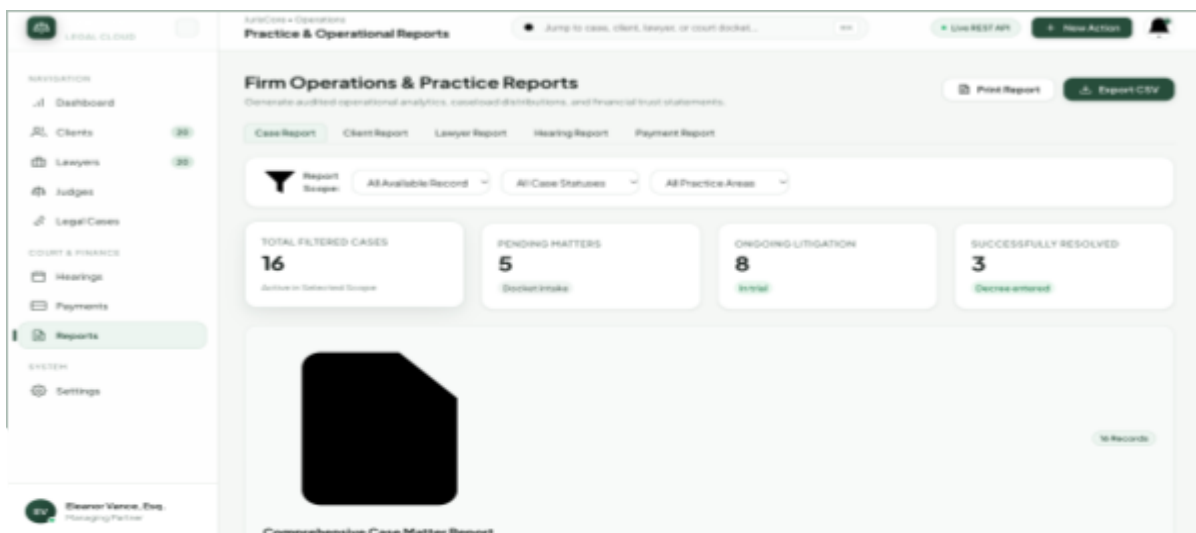


Figure — 11.8 Reports Screenshot.

12. Implementation Details

12.1 Technology Stack

Layer	Technology	Purpose
Frontend	HTML + CSS + JavaScript	Interactive and responsive user interface
UI Styling	Bootstrap 5 / custom CSS	Responsive layout and consistent visual design
Backend	Node.js + Express.js	REST API and application logic
Database Driver	mysql2	MySQL connectivity
Database	MySQL	Persistent relational data
Configuration	dotenv	Environment-based configuration
Development	VS Code + MySQL Workbench	Application and database development

12.2 Database Connectivity

The backend uses environment variables for database configuration and a MySQL connection pool for efficient database access. Express handles JSON requests and exposes REST endpoints to the frontend. The frontend communicates with the backend rather than connecting directly to MySQL.

```
DB_HOST=localhost
t DB_PORT=3306
DB_USER=root
DB_PASSWORD=your_password
DB_NAME=LegalCaseDB
PORT=5000
```

12.3 API Structure

Resource	Main Endpoints
Clients	/api/clients, /api/clients/:id
Lawyers	/api/lawyers, /api/lawyers/:id
Judges	/api/judges, /api/judges/:id
Cases	/api/cases, /api/cases/:id
Hearings	/api/hearings, /api/hearings/:id
Payments	/api/payments, /api/payments/:id
Dashboard	/api/dashboard/stats
Lawyer-Case	/api/works-on

12.4 Backend and Database Logic

Database operations should use parameterized queries for user-controlled values. Foreign-key relationships maintain referential integrity between clients, cases, hearings, judges, payments, lawyers, and case assignments. The backend should validate incoming data, return controlled errors, and provide JSON responses to the frontend. For multi-step operations, transactions may be used so related changes are committed or rolled back together.

```
const [rows] = await pool.query( 'SELECT * FROM Client ORDER BY client_id' );
```

13. Testing

Testing focuses on database connectivity, CRUD operations, relational queries, foreign-key consistency, frontend-backend communication, search/filtering, and dashboard data.

Test Case	Expected Result	Result
Database connection	Backend connects successfully to LegalCaseDB	Pass
View clients	Client records load in the UI	Pass
View lawyers	Lawyer records load correctly	Pass
View judges	Judge records load correctly	Pass
View cases	Cases load with linked client information	Pass
Create record	New record persists in MySQL	Pass
Update record	Updated values persist in MySQL	Pass
Delete record	Valid deletion persists subject to FK constraints	Pass
Hearing query	Case, judge and hearing data are correctly related	Pass
Payment query	Payment is displayed with client and case relationship	Pass
Lawyer assignment	Lawyer–case relationships are displayed correctly	Pass
Dashboard statistics	Values reflect database state	Pass
Error handling	Invalid operations return controlled errors	Pass

14. Conclusion and Future Enhancements

14.1 Conclusion

The Legal Case & Client Management System provides a structured relational solution for managing legal operations. The database separates client, lawyer, judge, case, hearing, payment, and lawyer–case assignment data into appropriate relations, while primary keys and foreign keys preserve relationships and data integrity. The connected backend and frontend provide practical CRUD access, case tracking, hearing management, payment tracking, search, and dashboard reporting.

The project demonstrates important DBMS concepts including relational modeling, ER design, normalization, DDL and DML, joins, aggregation, foreign-key constraints, and database-backed application development. It also provides a foundation that can be extended into a larger legal information management platform.

14.2 Future Enhancements

- User authentication and role-based access control.
- Advocate/lawyer workload and case-performance analytics.
- Automated hearing reminders and notifications.
- Document upload and case-document management.
- Online payment gateway integration.
- Advanced reporting and export to PDF/Excel.
- Cloud deployment and secure remote database access.
- Audit logs for tracking important record changes.
- Mobile-responsive enhancements and optional mobile application support.

15. References

-
- MySQL Documentation — relational database concepts, SQL, constraints, joins, and transactions.
 - Node.js Documentation — backend runtime environment and server-side JavaScript.
 - Express.js Documentation — REST API framework and middleware.
 - mysql2 Documentation — Node.js MySQL client and connection pooling.
 - Bootstrap Documentation — responsive interface components and layout.
 - Project database schema, SQL scripts, and implementation developed for the Legal Case & Client Management System.

16. Appendix: GitHub Repository

The complete source code, frontend, backend, database scripts, configuration files, and project documentation can be maintained in the following repository:

<https://github.com/gagansrivarma/DBMS-Course-Project.git> PLACE GITHUB LINK HERE

Suggested format: <https://github.com/>

— END OF PROJECT REPORT —